LAPORAN PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK
MODUL [5] – Relasi Antar Kelas

Rimbhi Putri Aulia Azzahra

103132400018

PS1DS-05-REG-01

PROGRAM STUDI SAINS DATA

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

2026

A. Dasar Teori

Relasi antar kelas merupakan hubungan antara dua kelas atau lebih. Relasi ini memungkinakan objek saling berhubungan dan bekerja sama untuk menjalankan fungsi tertentu tertentu dalam program. Terdapat beberapa jenis relasi antar kelas, diantaranya komposisi, agregasi dan asosiasi.

1. Komposisi (Composition)

   Merupakan relasi ketika suatu objek memiliki objek lain sebagai bagian internalnya, sehingga keberadaan objek bagain bergantung pada objek utamanya. Objek bagian biasanya dibuat di dalam constructor kelas utama.

2. Agregasi (Aggregation)

   Merupakan relasi ketika suatu objek menampung atau menggunakan objek lain yang dapat berdiri sendiri. Objek yang ditampung dibuat secara terpisah, kemudian dimasukkan ke dalam objek penampung.

3. Asosiasi (Association)

   Merupakan relasi antara dua objek independen yang saling berinteraksi tanpa memiliki satu sama lain. Interaksi dapat dilakukan melalui parameter method.

   Ketiga relasi tersebut digunakan untuk merancang program yang memiliki hubungan antarkelas sesuai kebutuhan. Dalam penerapannya, enkapsulasi juga digunakan agar data di dalam kelas tidak dapat diakses atau diubah secara langsung dari luar kelas. Data disimpan menggunakan field private dan diakses melalui method yang sesuai.

B. Guided

1. Asosiasi

   - Mahasiswa.java

```java
package com.rimbhi.guided.guided01;

public class Mahasiswa {
     private String nim;
     private String nama;
     private int nilai;

     public void setNama(String nama){
        this.nama = nama;
    }
    public void setNim(String nim){
        this.nim = nim;
    }
```

```
    public void setNilai(int nilai){
        this.nilai = nilai;
    }

    // Getter
    public int  getNilai(){
        return this.nilai;
    }

    public String getNama(){
        return this.nama;
    }
    public String getNim(){
        return this.nim;
    }
}
```

- Dosen.java

```
package com.rimbhi.guided.guided01;

public class Dosen {
     private String kodeDosen;
     private String namaDosen;
     //Setter
     public void setKodeDosen(String kodeDosen){
        this.kodeDosen = kodeDosen;     }
    public void setNamaDosen(String namaDosen){
        this.namaDosen = namaDosen;
}
    //Getter
    public String getKodeDosen(){
            return this.kodeDosen;
}
    public String getNamaDosen(){
            return this.namaDosen;
}
    public void giveScore(Mahasiswa s, int nilai){
        s.setNilai(nilai);
}
     public int getScore(Mahasiswa s){
        return s.getNilai();
}
 public static void main(String[] args) {
    Mahasiswa m = new Mahasiswa();
    m.setNim("130118383");
    m.setNama("Budi");
```

```
    Dosen d = new Dosen();
    d.giveScore(m, 90);
    System.out.println("Nim :"+m.getNim());
    System.out.println("Nama :"+m.getNama());
    System.out.println("Nilai :"+d.getScore(m));
}
}
```

Output:

```
Nim :130118383
Nama :Budi
Nilai :90
```

2. Agregasi
    - Mahasiswa.java

```
package com.rimbhi.guided.agregasi;

public class Mahasiswa{
    private String NIM, Nama;

    public Mahasiswa(String NIM, String Nama){
        this.NIM = NIM;
        this.Nama = Nama;
    }

    // getter
    public String getNIM(){
        return NIM;
    }
    public String getNama(){
        return Nama;
    }

    // setter
    public void setNIM(String NIM){
        this.NIM = NIM;
    }
    public void setNama(String Nama){
        this.Nama = Nama;
    }
}
```

    - Jurusan.java

```
package com.rimbhi.guided.agregasi;
```

```
public class Jurusan {
    private static int jumlahMahasiswa;
    private String kodeJurusan, namaJurusan;
    private Mahasiswa[] daftarMahasiswa = new Mahasiswa[32];

    // constractor
    public Jurusan(String kodeJurusan, String namaJurusan){
        this.kodeJurusan = kodeJurusan;
        this.namaJurusan = namaJurusan;
    }
    // behavior
    public void addMahasiswa(Mahasiswa m){
        this.daftarMahasiswa[jumlahMahasiswa] = m;
        jumlahMahasiswa++;
    }
    public void displayMahasiswa(){
        System.out.println("Kode jurusan: " +  this.kodeJurusan);
        System.out.println("Nama jurusan: " + this.namaJurusan);

        System.out.println("Daftar mahasiswa: ");

        for(int i = 0; i<jumlahMahasiswa; i++){
            System.out.println(i+1 + ". " +
daftarMahasiswa[i].getNIM() + " " + daftarMahasiswa[i].getNama());
        }

    }
    // getter
    public String getkodeJurusan(){
        return kodeJurusan;
    }
    public String getnamaJurusan(){
        return namaJurusan;
    }

    // setter
    public void setkodeJurusan(String kodejurusan){
        this.kodeJurusan = kodejurusan;
    }
    public void setnamaJurusan(String namaJurusan){
        this.namaJurusan = namaJurusan;
    }
}
```

- Main.java

```
package com.rimbhi;

import com.rimbhi.guided.agregasi.*;
```

```java
import com.rimbhi.guided.komposisi.*;

public class Main {
    public static void main(String[] args) {
        runagregasi();
        //runkomposisi();
    }


    public static void rungided01(){}

    public static void runagregasi(){
        Mahasiswa m1 = new Mahasiswa("103112400118", "Rimbhi");
        Mahasiswa m2 = new Mahasiswa("103112400149", "John");

        Jurusan prodi = new Jurusan("SD","Sains Data");

        prodi.addMahasiswa(m1);
        prodi.addMahasiswa(m2);

        prodi.displayMahasiswa();

    }

    public static void runkomposisi(){
        Komputer pc = new Komputer("Intel Core i5",3200,"LG 24
Inch","Logitech","Wireless");
        pc.displaySpecKomputer();
    }
}
```

Output:

```
Kode jurusan: SD
Nama jurusan: Sains Data
Daftar mahasiswa:
1. 103112400118 Rimbhi
2. 103112400149 John
```

3. Komposisi
    - Processor.java

```java
package com.rimbhi.guided.komposisi;

public class Processor {
    private String brand;
    private int kecepatan;

    //constructor
```

```
    public Processor(String brand,int kecepatan){
        this.brand = brand;
        this.kecepatan = kecepatan;
    }

    //behaviour
    public void displaySpec(){
        System.out.println(this.brand + ", " + this.kecepatan);
    }
}
```

- Mouse.java

```
package com.rimbhi.guided.komposisi;

public class Mouse {
    private String brand, tipe;

    public Mouse(String brand, String tipe){
        this.brand = brand;
        this.tipe = tipe;
    }

    public void displaySpec(){
        System.out.println(this.brand + ", " + this.tipe);
    }
}
```

- Monitor.java

```
package com.rimbhi.guided.komposisi;

public class Monitor {
      private String brand;

    public Monitor(String brand){
        this.brand = brand;
    }

    public void DisplaySpec() {
        System.out.println(this.brand);
    }
}
```

- Komputer.java

```
package com.rimbhi.guided.komposisi;
```

```java
public class Komputer {
    private Processor processor;
    private Monitor monitor;
    private Mouse mouse;

    public Komputer(String brandCpu, int kecepatanCpu, String
brandMonitor, String brandMouse, String tipeMouse){
        this.processor = new Processor(brandCpu, kecepatanCpu);
        this.monitor = new Monitor(brandMonitor);
        this.mouse = new Mouse(brandMouse, tipeMouse);
    }

    public void displaySpecKomputer() {
        System.out.println("=== SPESIFIKASI KOMPUTER ===");
        this.processor.displaySpec();
        this.monitor.DisplaySpec();
        this.mouse.displaySpec();
    }
}
```

- Main.java

```java
package com.rimbhi;

import com.rimbhi.guided.agregasi.*;
import com.rimbhi.guided.komposisi.*;

public class Main {
    public static void main(String[] args) {
        //runagregasi();
        runkomposisi();
    }


    public static void rungided01(){}

    public static void runagregasi(){
        Mahasiswa m1 = new Mahasiswa("103112400118", "Rimbhi");
        Mahasiswa m2 = new Mahasiswa("103112400149", "John");

        Jurusan prodi = new Jurusan("SD","Sains Data");

        prodi.addMahasiswa(m1);
        prodi.addMahasiswa(m2);

        prodi.displayMahasiswa();

    }
```

```
    public static void runkomposisi(){
        Komputer pc = new Komputer("Intel Core i5",3200,"LG 24
Inch","Logitech","Wireless");
        pc.displaySpecKomputer();
    }
}
```

Output:

```
=== SPESIFIKASI KOMPUTER ===
Intel Core i5, 3200
LG 24 Inch
Logitech, Wireless
```

C. Unguided

1. Soal Unguided

- RincianHarga.java

```
package com.rimbhi.unguided.cafe;

public class RincianHarga {
    private double hargaDasar;
    private double pajak;

    public RincianHarga(double hargaDasar){
        this.hargaDasar = hargaDasar;
        this.pajak = hargaDasar * 0.1;
    }

    public double getHargaTotal(){
        return hargaDasar + pajak;
    }

    public double getHargaDasar(){
        return hargaDasar;
    }

    public double getPajak(){
        return pajak;
    }
}
```

- ItemPesanan.java

```
package com.rimbhi.unguided.cafe;

public class ItemPesanan {
    private String namaItem;
    private RincianHarga rincian;
```

```
    public ItemPesanan(String namaItem, double hargaDasar){
        this.namaItem = namaItem;
        this.rincian = new RincianHarga(hargaDasar);
    }

    public String getNamaItem(){
        return namaItem;
    }

    public RincianHarga getRincian(){
        return rincian;
    }
}
```

- Pesanan.java

```
package com.rimbhi.unguided.cafe;

public class Pesanan {
    private int nomorMeja;
    private ItemPesanan[] daftarItem;
    private int jumlahItem;

    public Pesanan(int nomorMeja, int kapasitas){
        this.nomorMeja = nomorMeja;
        this.daftarItem = new ItemPesanan[kapasitas];
        this.jumlahItem = 0;
    }

    public void tambahItem(ItemPesanan item) {
        if (jumlahItem < daftarItem.length){
            daftarItem[jumlahItem] = item;
            jumlahItem++;
        } else {
            System.out.println("Pesanan penuh, tidak bisa
menambah item lagi.");
        }
    }

    public void tampilkanPesanan() {
    System.out.println("=== DAFTAR PESANAN CAFE ===");
    System.out.println("Nomor Meja: " + nomorMeja);
    for (int i = 0; i < jumlahItem; i++) {
        ItemPesanan item = daftarItem[i];
        RincianHarga r = item.getRincian();
        System.out.println((i + 1) + ". " +
item.getNamaItem()
                + " - Rp " + r.getHargaTotal()
```

```
                + " (Harga: " + r.getHargaDasar()
                + ", Pajak: " + r.getPajak() + ")");
    }
}

    public int getNomorMeja(){
        return nomorMeja;
    }

    public ItemPesanan[] getDaftarItem(){
        return daftarItem;
    }

    public int getJumlahItem(){
        return jumlahItem;
    }
}
```

- Kasir.java

```
package com.rimbhi.unguided.cafe;

public class Kasir {
    private String namaKasir;

    public Kasir(String namaKasir) {
        this.namaKasir = namaKasir;
    }

    public void prosesPembayaran(Pesanan pesanan){
        double totalSubtotal = 0;
        double totalPajak = 0;

        ItemPesanan[] daftar = pesanan.getDaftarItem();
        for (int i = 0; i < pesanan.getJumlahItem(); i++){
            RincianHarga r = daftar[i].getRincian();
            totalSubtotal += r.getHargaDasar();
            totalPajak += r.getPajak();
        }

        double totalBayar = totalSubtotal + totalPajak;

        System.out.println("\n=== STRUK PEMBAYARAN KASIR ===
");
        System.out.println("Kasir Bertugas : " + namaKasir);
        System.out.println("Pesanan        : Meja " +
pesanan.getNomorMeja());
        System.out.println("Jumlah Item    : " +
pesanan.getJumlahItem());
```

```
        System.out.println("Total Subtotal : Rp " +
totalSubtotal);
        System.out.println("Total Pajak    : Rp " +
totalPajak);
        System.out.println("Total Bayar    : Rp " +
totalBayar);
        System.out.println("============================");

    }
}
```

- Main.java

```
package com.rimbhi.unguided.cafe;

public class Main {
    public static void main(String[] args){
        Kasir kasir = new Kasir("Budi");

        ItemPesanan item1 = new ItemPesanan("Iced
Americano", 18000);
        ItemPesanan item2 = new ItemPesanan("Beef
Croissant", 25000);
        ItemPesanan item3 = new ItemPesanan("Iced
Americano", 22000);

        Pesanan pesanan = new Pesanan(4, 5);

        pesanan.tambahItem(item1);
        pesanan.tambahItem(item2);
        pesanan.tambahItem(item3);

        pesanan.tampilkanPesanan();

        kasir.prosesPembayaran(pesanan);
    }
}
```

Output:

```
=== DAFTAR PESANAN CAFE ===
Nomor Meja: 4
1. Iced Americano - Rp 19800.0 (Harga: 18000.0, Pajak: 1800.0)
2. Beef Croissant - Rp 27500.0 (Harga: 25000.0, Pajak: 2500.0)
3. Iced Americano - Rp 24200.0 (Harga: 22000.0, Pajak: 2200.0)

=== STRUK PEMBAYARAN KASIR ===
Kasir Bertugas : Budi
Pesanan        : Meja 4
Jumlah Item    : 3
Total Subtotal : Rp 65000.0
Total Pajak    : Rp 6500.0
Total Bayar    : Rp 71500.0
==========================
```

Penjelasan:

- Pada bagian class RincianHarga menyimpan rincian harga sebuah item (harga dasar dan pajak) terdapat 3 method ada (getHargaTotal, gethargaDasar, dan getPajak) semua field di kelas ini bersifat private jadi tidak bisa di akses langsung dari luar.
- Lalu class ItemPesanan: untuk mewakili satu item pesanan, terdiri dari 2 method ada getNamaItem dan getRincian, ketika ItemPesanan dihapus maha RincianHarga juga akan ikut hilang (siklus hidup terikat) dan field rincian tidak bisa diganti dari luar karena tidak ada setter.
- Class Pesanan: untuk menampung banyak ItemPesanan untuk satu meja dengan relasi agregasi, dimana item dibuat di luar. Class ini terdiri dari beberapa Method diantaranya ada tambahItem(ItemPesanan item), tampilkanPesanan() mencetak header dan nomor meja, lalu melakukan loop hanya sampai jumlahItemsupaya slot kosong tidak ikut tercetak, dan setiap item ditampilkan pakai println agar rapi per baris. Terakhir, ada tiga getter getNomorMeja(), getDaftarItem(), dan getJumlahItem() yang dipakai Kasir untuk menghitung total bayar. Intinya, Pesanan hanyalah wadah yang menampung ItemPesanan dari luar tanpa membuat atau menghapusnya, sehingga siklus hidup item tetap independen.
- Pada class Kasir ini digunakan untuk memproses pembayaran sebuah pesanan, dengan menggunakan method prosesPembayaran, di dalam method ini, terdapat perulangan yang digunakan untuk menjumlahkan seluruh harga item, serta proses perhitungan total bayar akhir yang diperoleh dari penjumlahan subtotal dan pajak.

D. Kesimpulan

Relasi antar kelas menentukan tingkat keterikatan dan ketergantungn antar objek. Komposisi bersifat terikat mutlak karena objek dibuat langsung di dalam dan ikut hilang jika objek utama dihapus. Sedangkan agregasi bersifat terikat bebas karena objek dibuat terpisah di luar sehingga masih bisa berdiri sendiri. Sementara itu, asosiasi hanya berupa interaksi sesaat ketika objek saling memanfaatkan fungsi melalui parameter method tanpa memiliki hubungan kepemilikan.